

Assignment 2: Coding Basics

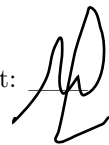
Alexis Clay

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.
7. Initial here to acknowledge that you did not use AI at all in completing this assignment:



Basics, Part 1

1. Use R’s `seq()` function to create a sequence of numbers from 100 to 333, increasing by threes. Assign this sequence a variable name.
2. Compute the *mean* of this sequence, assigning this values its own variable name.
3. Compute the *standard deviation* (`sd()`) of this sequence, assigning this values its own variable name.
4. Display the the mean minus the standard deviation as well as the mean plus the standard deviation.
5. Insert comments in your code to describe what you are doing.

```
#1.  
  
#using seq(from, to, by) function to create a sequence and assigning it a variable.  
question1 <- seq(100,333,3)  
  
#2.  
  
#calculating the mean of the sequence using the variable I assigned it in #1.  
meanquestion1 <- mean(question1)  
  
#3.
```

```
#calculating the sd of the sequence using the variable I assigned it in #1.  
sdquestion1 <- sd(question1)
```

```
#4.  
#this is the mean of sequence 1 minus its sd  
meanquestion1 - sdquestion1
```

```
## [1] 147.5184
```

```
#this is the mean of sequence 1 plus its sd  
meanquestion1 + sdquestion1
```

```
## [1] 283.4816
```

Basics, Part 2

6. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
7. Label each vector with a comment on what type of vector it is.

```
#this is a character vector. I can check using class(studentnames) in the console.  
studentnames <- c("Marie Curie", "Albert Einstein", "Enrico Fermi", "Alan Turing")  
  
#this is a numeric vector  
testscores <- c(92, 73, 93, 89)  
  
scholarship <- c(TRUE,FALSE,FALSE,TRUE)
```

8. Combine each of the vectors into a data frame. Assign the data frame an informative name.

```
#combining my vector variables into a frame using data.frame(vector variables)  
geniusgraduatesframe <- data.frame(studentnames,testscores, scholarship)
```

9. Label the columns of your data frame with informative titles.

```
names(geniusgraduatesframe)<-c("Name", "Score","Scholarship")
```

10. QUESTION: How is this data frame different from a matrix?

Answer: The columns of a data frame are comprised of vectors that can be different types. Conversely, all the data in a matrix must be of the same type.

Basics, Part 3

11. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, it returns the word “Pass”; otherwise it returns the word “Fail”.

```
test1_score <- function(x){  
  if (x>50){  
    print("Pass")  
  }else{  
    print("Fail")  
  }  
}
```

12. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.

```
test2_score<- function(x){  
  ifelse(x > 50, "Pass", "Fail")}
```

13. Run both functions using the value 54 as the input

```
test1_grade <- test1_score(54) #prints output automatically
```

```
## [1] "Pass"
```

```
test2_grade <- test2_score(54) #output in console
```

14. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#run with if...else
```

```
# test1_score(testscores) this returns the error: Error in if (testscores > 50) { : the condition has l
```

```
test2_score(testscores) #this returns "pass" four times
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

```
{r basics 3} #11. Create a function using if...else
```

```
myage <- function(x){  
  if (x < 30){  
    print("This is the decade of the existential crisis.")  
  }else if (x >=30 & x < 40){  
    print("30s is the new 20s!")  
  }else{  
    print("Old? Age is just a numeric variable!")  
  }  
}  
  
myage(62)
```

```
## [1] "Old? Age is just a numeric variable!"
```

#12. Create a function using ifelse()

```
siblingages <- c(3, 7, 13, 16)

siblingstatus <- function(x){
  ifelse(x>12, "Teenager", "Child")
}

youngestage <- siblingstatus(3)
```

#13a. Run the first function with the value 54

```
fiftyfour<- myage(54)
```

```
## [1] "Old? Age is just a numeric variable!"
```

#13b. Run the second function with the value 54

```
siblingstatus(54)
```

```
## [1] "Teenager"
```

#14a. Run the first function with the vector of test scores

```
# myage(testscores)
```

#14b. Run the second function with the vector of test scores

```
siblingstatus(testscores)
```

```
## [1] "Teenager" "Teenager" "Teenager" "Teenager"
```

15. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”; it’s ok here if an AI response is presented in the search response.)

Answer:

An `if..else` statement is built to evaluate true or false for a single element. Only an `ifelse()` statement works to run a vector because it tests each element independently, which I imagine will make it helpful for analyzing large datasets efficiently.

NOTE Before knitting, you’ll need to comment out the call to the function in Q14 that does not work. (A document can’t knit if the code it contains causes an error!)